

Graph-Driven Meta-Agents: Integrating Model Context Protocol and LangGraph for Autonomous Agent Generation

PROJECT REPORT

Submitted by

BL.EN.U4AIE22019 Gamidi Rohan

BL.EN.U4AIE22059 Tejaswi Muppala

BL.EN.U4AIE22065 V S S Ashish Babu

BL.EN.U4AIE22066 Vinitha Chowdary A

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

**COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE)**



AMRITA SCHOOL OF COMPUTING, BENGALURU

AMRITA VISHWA VIDYAPEETHAM

BENGALURU 560035

DECEMBER 2025

ACKNOWLEDGEMENTS

The satisfaction that accompanies successful completion of any task would be incomplete without mention of people who made it possible, and whose constant encouragement and guidance have been source of inspiration throughout the course of this project work.

We offer our sincere pranams at the lotus feet of **"AMMA", MATA AMRITANAN-DAMAYI DEVI** who showered her blessing upon us throughout the course of this project work.

We owe our gratitude to **Prof. Manoj P.**, Director, Amrita Vishwa Vidyapeetham Bengaluru Campus. We would like to place our heartfelt gratitude to **Dr. Gopalakrishnan E.A.**, Principal, Amrita School of Computing and Amrita School of Artificial Intelligence, Bengaluru for his valuable support and inspiration.

It is a great pleasure to express our gratitude and indebtedness to our project guide **Dr. S Santhanalakshmi, Assistant Professor (Sl. Gd.)** , School of Artificial Intelligence, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Bengaluru for his valuable guidance, encouragement, moral support, and affection throughout the project work.

We would like to express our gratitude to project panel members for their suggestions, encouragement, and moral support during the process of project work and all faculty members for their academic support. Finally, we are forever grateful to our parents, who have loved, supported, and encouraged us in all our endeavors.

ABSTRACT

Graph-Driven Meta-Agents: Integrating Model Context Protocol and LangGraph for Autonomous Agent Generation tackles the growing complexity of the development of modern AI agents, in which individual agents require a manual setup of various tools, APIs, documentation access, and external services, is addressed in Graph-Driven Meta-Agents: Model Context Protocol and LangGraph. Current AI assistants and coding models are reactive, that is, they produce output or generate code, but they do not plan workflows, check their own output, or make new agents, without human intervention by the developer. This drawback leads to a lot of development work, manual engineering several times and systems that are prone to failure as requirements change. This work aims at empowering an AI agent to act at a meta-level, plan, build, and optimize other agents by means of systematic thinking, documentation retrieval and feedback loops. The system helps in hallucination reduction, enhancing the reliability, and developing agents in a cost competition and time-efficient way through the alternative of single reactive agent to coordinated agentic workflow development. Graph-Driven Meta-Agents: Integrating Model Context Protocol and LangGraph for Autonomous Agent Generation applies a layered and modular methodology that integrates Pydantic AI for structured agent definitions, LangGraph for multi-agent orchestration, Supabase vector database for Retrieval-Augmented Generation over technical documentation, and the Model Context Protocol (MCP) to grant agents secure access to external tools and services. The best results achieved so far include reliable autonomous generation of new agents, prompt and tool refinement through specialized sub-agents, and successful reuse of prebuilt MCP servers and tool libraries to reduce redundant work and lower hallucination rates. In comparison to existing AI systems like Copilot and single LLM assistants that only generate code passively, this work demonstrates an active, planning-based meta-agent that not only generates agents but also evaluates, improves, and configures them, offering higher scalability, lower manual effort.

Contents

ACKNOWLEDGEMENTS	1
ABSTRACT	2
LIST OF FIGURES	7
LIST OF TABLES	8
1 INTRODUCTION	9
1.1 Introduction to the Problem Domain	9
1.1.1 What is the Problem?	9
1.1.2 Why is That a Problem? How Does it Affect People and Society?	9
1.2 Motivation	10
1.3 What is Done to Solve the Problem?	10
1.4 What is Proposed to be Done?	11
1.5 Novelty and UN SDG Alignment	11
1.5.1 What is the Novelty?	11
1.5.2 UN SDG Alignment	12
1.6 Objectives	12
1.7 Organization of the Report	12
2 LITERATURE REVIEW	14
2.1 Background Research	14
2.2 Related Work	15
2.2.1 Autonomous Agents and Multi-Agent Collaboration	15
2.2.2 Agentic Workflow and Orchestration Frameworks	15
2.2.3 Interoperability and Protocol Standards	16
2.2.4 Existing System Limitations	16

2.3	Comparative Analysis	17
3	SYSTEM SPECIFICATIONS	19
3.1	Software Requirements	19
3.2	Hardware Requirements	21
4	SYSTEM DESIGN	23
4.1	System Architecture	23
4.1.1	User Interaction Layer	23
4.1.2	Initial Query Processing	23
4.1.3	Agent Orchestration Layer	24
4.1.4	Retrieval-Augmented Generation (RAG)	24
4.1.5	Tool Invocations via MCP (Model Context Protocol)	25
4.1.6	LLM Reasoning	25
4.1.7	Offline Ingestion Pipeline	25
4.2	Low-Level Design	26
4.2.1	User Interface Layer	27
4.2.2	Backend Services	27
4.2.3	Graph-Based Agent Workflows	27
4.2.4	External Tool Integration	28
4.2.5	Memory Storage and State Management	28
4.2.6	Runtime Environment	28
4.2.7	Error Handling and Concurrency	28
4.2.8	Extension Points	29
4.2.9	Database Schema Overview	29
4.2.10	Indexes and Constraints	30
4.2.11	Search Functionality	30
4.2.12	Security Policies	31
4.2.13	Embedding Update and Maintenance	31
4.2.14	Entity Relationships and Data Flow	31
5	SYSTEM IMPLEMENTATION	32
5.1	Modules Used with Description	32
5.1.1	Streamlit UI Design	32

5.1.2	Graph Service Design (FastAPI)	33
5.1.3	LangGraph Workflow Engine	34
5.1.4	Pydantic AI Coder Component	35
5.1.5	Refiner Agents	35
5.1.6	Advisor Agent	35
5.1.7	Agent Tools & Resources	35
5.1.8	MCP Server & Integration	35
5.1.9	Utilities & Database Integration	36
5.1.10	Workbench Runtime Files	36
5.1.11	Docker Deployment	36
5.1.12	Configuration & Environment	36
5.1.13	APIs and Contracts	36
5.1.14	Data Models	36
5.2	Concurrency, Async Behavior & Resource Management	37
6	SYSTEM TESTING	38
6.1	Testing Strategy	38
6.2	Testing Strategy	38
6.3	Test Cases	40
6.4	Performance Testing	41
6.4.1	Performance Metrics	41
6.4.2	CI and Automation	42
6.4.3	Security Measures	42
7	RESULTS AND ANALYSIS	43
7.1	Experimental Setup	43
7.1.1	System Implementation	43
7.1.2	Dataset and Knowledge Base	44
7.1.3	Test Queries	45
7.1.4	Evaluation Metrics	45
7.2	Results	45
7.2.1	Accuracy and Relevance	45
7.2.2	Example Query and Output	46

7.3	Analysis	47
7.3.1	Performance Summary	47
7.3.2	Key Findings	47
7.3.3	Limitations and Areas for Improvement	48
7.3.4	Implications for Deployment	48
8	CONCLUSION AND FUTURE SCOPE	50
8.1	Conclusion	50
8.2	Future Enhancements	52
8.3	Applications	53

List of Figures

4.1	System Architecture Diagram	24
4.2	Low Level System Design	26
4.3	Database Schema for site_pages table	30
7.1	Streamlit UI - Main interface showing Agent Builder introduction and setup guide	43
7.2	Streamlit UI - Environment configuration section showing required variables and setup steps	44

List of Tables

2.1	Comparison of Existing Approaches and Proposed Solution Features	17
6.1	Sample Test Cases	41
7.1	Query performance against expected ground truth	46

Chapter 1

INTRODUCTION

1.1 Introduction to the Problem Domain

Modern artificial intelligence applications rely increasingly on **autonomous agents** to interact with APIs, access external tools, retrieve information from documentation, and perform complex workflows. However, building such agents is still a largely **manual and time-consuming** process. Developers must repeatedly configure every agent with tool integrations, database access, authentication, memory models, and reasoning steps. Even after deployment, most agents remain **reactive**, meaning they can only respond to prompts but cannot modify themselves, improve their logic, or create new agents for new tasks. As systems become more complex and distributed, this limitation results in high development cost, difficulty in scaling agent functionality, and rapid system fragility when requirements change.

1.1.1 What is the Problem?

The central problem is that existing AI agents are not autonomous in the **development process** they rely heavily on manual engineering. Building an intelligent agent still requires step-by-step coding for reasoning, memory integration, environment setup, and external tool connections. Moreover, most agents lack the ability to **self-refine**, **reuse tools**, or plan their own workflows.

1.1.2 Why is That a Problem? How Does it Affect People and Society?

This constraint leads to significant **development bottlenecks**, especially for small teams, startups, educators, and researchers who may lack the resources to build complex AI systems. The growing demand for automation in healthcare, education, manufacturing, and public services

cannot be fulfilled at scale if intelligent systems are hard to build and brittle to maintain. Furthermore, manual setup increases the likelihood of errors like hallucinations, security leaks, and system failures. As a result, many potentially transformative applications of AI remain **inaccessible or unaffordable** to the wider society.

1.2 Motivation

Earlier attempts to overcome this challenge involved enhancing individual agents using techniques such as **prompt engineering**, **Retrieval-Augmented Generation (RAG)**, and **plugin-based tool integrations**. While these methods improved local accuracy, they did not eliminate the need for human developers to remain **in the loop**. Agents were still incapable of autonomously planning workflows, improving themselves, or spawning new agents to solve emerging tasks.

The motivation for this work stems from the need to make **AI agent development self-directed**, scalable, and reusable. It is about enabling agents that are not only consumers of knowledge but **producers of new capability** agents that can think, reason, revise, and expand.

1.3 What is Done to Solve the Problem?

This work introduces a **graph-driven meta-agent system** that redefines how intelligent agents are built. Instead of designing each agent manually, this system allows a **meta-agent** to:

- Understand the problem through reasoning,
- Retrieve relevant documentation via semantic search,
- Generate code using structured AI coders,
- Integrate existing tools securely,
- Iterate and refine based on user feedback, and
- Export new agents with clear scope and behavior.

This is accomplished through a **LangGraph**-based workflow where each node is a specialized agent such as a reasoner, advisor, coder, or refiner working together to autonomously generate functional agents.

1.4 What is Proposed to be Done?

This project, titled **Graph-Driven Meta-Agents: Integrating Model Context Protocol and LangGraph for Autonomous Agent Generation**, proposes the following solution:

- Use **LangGraph** for orchestrating multi-agent workflows where agents pass control and context to each other.
- Employ **Pydantic AI** to define structured agent responses and maintain consistency across different modules.
- Use **Supabase** as a semantic vector database to perform context-aware documentation retrieval.
- Integrate the **Model Context Protocol (MCP)** to allow secure, pluggable, and reusable tool definitions.
- Provide a **Streamlit-based UI** to guide users through environment setup, documentation crawling, and agent creation.

This system is designed to be extensible, configurable, and general-purpose, with the ability to operate across domains without requiring deep technical configuration.

1.5 Novelty and UN SDG Alignment

1.5.1 What is the Novelty?

The **novelty** of this work lies in its transformation of the traditional reactive AI agent into a **self-improving, context-aware, graph-driven meta-agent**. Unlike tools such as GitHub Copilot, which assist a human in coding, the proposed system can **plan, generate, and refine** new agents without requiring manual configuration. It reuses existing components, reduces hallucinations through memory and document grounding, and integrates tools securely using MCP.

Furthermore, it incorporates **structured feedback loops** via refiners and **reasoning components** that ensure each agent is aligned to its scope and goal. It evolves over multiple versions (v1 to v6), with each stage introducing greater levels of autonomy and intelligence.

1.5.2 UN SDG Alignment

This project supports **UN Sustainable Development Goal (SDG) 9: Industry, Innovation, and Infrastructure** by:

- Accelerating the **automation of software workflows**,
- Making AI innovation more **affordable and accessible**,
- Reducing reliance on large technical teams,
- Improving **software development efficiency** in startups, public institutions, and educational settings.

1.6 Objectives

The specific objectives of this work are:

- **Objective 1:** Automate agent creation using a graph-based meta-agent system.
- **Objective 2:** Enable agents to self-refine through iterative reasoning, coding, and feedback loops.
- **Objective 3:** Integrate reusable tools using secure model-context bindings through MCP.
- **Objective 4:** Provide a visual, low-barrier interface for configuration and usage through Streamlit.
- **Objective 5:** Use a vector database (Supabase) for scalable memory-aware retrieval.
- **Objective 6:** Evaluate improvements through staged agent versions (v1 to v6).

1.7 Organization of the Report

The rest of the report is structured as follows:

- **Chapter 2: Literature Survey** – A review of existing tools, frameworks, and prior works in autonomous agents, orchestration frameworks, and structured prompting.

- **Chapter 3: System Specifications** – Details functional requirements, system design constraints, and software/hardware needs.
- **Chapter 4: System Design** – Covers both high-level and low-level system architecture, component interactions, data models, and workflows.
- **Chapter 5: System Implementation** – Describes in-depth the system's actual implementation, including versions from v1 to v6.
- **Chapter 6: System Testing** – Discusses system testing strategies: unit, integration, performance, and acceptance testing.
- **Chapter 7: Results and Analysis** – Provides output evaluations, feature comparisons, and performance observations.
- **Chapter 8: Conclusion and Future Scope** – Envisions future enhancements including real-time integration, enterprise extensions, and IDE plugin support, Summarizes the key contributions and societal relevance of the system.

Chapter 2

LITERATURE REVIEW

This chapter presents a comprehensive review of existing literature related to the project domain. The project, **Graph-Driven Meta-Agents: Integrating Model Context Protocol and LangGraph for Autonomous Agent Generation**, sits at the intersection of Large Language Models (LLMs), autonomous agents, multi-agent systems, Retrieval-Augmented Generation (RAG), and structured workflow orchestration.

2.1 Background Research

The foundational concepts for this project are rooted in Large Language Models (LLMs) and the architecture of autonomous agents. LLMs serve as the core reasoning engine for these agents, enabling capabilities such as profiling, planning, and action generation [1]. Early autonomous agent systems like AutoGPT and BabyAGI demonstrated the potential of LLMs to break down complex goals into executable steps but struggled with limitations like uncontrolled recursive loops, goal finalization, and frequent hallucinations.

A critical component to address the LLM’s inherent knowledge cutoff and hallucination issues is Retrieval-Augmented Generation (RAG) [22]. RAG enhances the LLM’s performance by dynamically retrieving and incorporating external information from a knowledge base (like a vector store), thereby improving accuracy, relevance, and factuality across various tasks [7, 21]. The performance of RAG heavily relies on the quality of vector embeddings and the retrieval mechanism [6, 13]. The use of graph databases alongside vector embeddings has also been explored to provide better contextual responses than traditional keyword-based systems [8, 34].

Another key background area is Multi-Agent Systems (MAS) and Workflow Orchestration. MAS leverage multiple specialized agents that collaborate to solve a problem [2, 26]. For the

agents to work effectively, a structured agent workflow is essential, analyzing planning, collaboration, and tool integration patterns [4, 5]. The limitations in older, rigid agent frameworks necessitate more flexible, graph-based coordination mechanisms to manage the complexity of multi-agent interactions and dynamic decision-making [35, 30].

2.2 Related Work

Existing research highlights both the rapid progress and the persistent challenges in developing robust autonomous agents.

2.2.1 Autonomous Agents and Multi-Agent Collaboration

Several recent surveys comprehensively define the field of Agentic AI, emphasizing core principles such as autonomy, goal-orientation, and adaptability [32, 25, 29]. Research has categorized these systems based on their interfaces, collaboration modes, and decision structures, noting that multi-agent setups generally improve reasoning diversity but face stability and evaluation challenges [2]. Specialized multi-agent systems have been proposed for complex tasks such as autonomous code generation [10, 11] and multimodal information extraction [24]. The concept of self-evolving agents, capable of adapting and improving over time, represents a major future direction [33].

2.2.2 Agentic Workflow and Orchestration Frameworks

The need for structured, dynamic workflows has led to the development of several prominent frameworks. LangChain is widely reviewed for its flexible connectors and vectorstore integrations [14], but it is noted for its limitations in evaluation and privacy. LangGraph has emerged as a key tool for orchestration, enabling the composition of agent workflows as directed graphs, which is seen as superior for scalable and inspectable agentic pipelines [35, 30]. Other advanced frameworks, such as DSPy, focus on compiling declarative language model calls into self-improving, optimized modules [19], while zero-code solutions like AutoAgent aim to democratize agent construction [36]. Research also explores how graph structures can enhance planning, memory, and reliability for LLM agents [34].

2.2.3 Interoperability and Protocol Standards

A significant challenge in MAS is interoperability and coordination. Existing literature points to the necessity of standardized communication protocols across different agent ecosystems [28, 31]. The Agent-to-Agent (A2A) protocol has been detailed as a standard for secure, interoperable communication [27]. More recently, the Model Context Protocol (MCP) has been proposed to solve context fragmentation and coordination issues, particularly in enterprise and scientific applications [23, 17]. MCP defines a structured way for agents to understand, discover, and invoke external tools and share information, which is crucial for building robust, scalable systems.

2.2.4 Existing System Limitations

Autonomous agent systems such as AutoGPT, ReAct, BabyAGI, LangChain Agents, and CAMEL have pushed the boundaries of language model-driven automation. However, these systems suffer from several well-documented limitations. AutoGPT, though popular, struggles with uncontrolled recursive loops, lack of goal finalization logic, and frequent hallucination of tasks or APIs. BabyAGI attempts task prioritization and memory-based iteration, but its flat memory model and rigid task chaining limit its adaptability. LangChain Agents and ReAct made progress in tool usage and reasoning-action loops respectively, yet neither offers true transparency in decision flows nor modular control over multi-agent interactions. Most of these models are constrained by brittle prompting, limited state persistence, and opaque inner workings, making them hard to debug, scale, or align with real-world user intentions. These existing systems also lack a structured way to integrate third-party tools and services based on a universal communication protocol. For example, invoking a web search or triggering code execution is often hardcoded or tied to a brittle API schema. This leads to poor extensibility when new tools need to be plugged in. Additionally, most existing agent frameworks have no built-in mechanism to visualize their planning structure or enable runtime introspection, which creates barriers for developers and researchers who want to monitor, fine-tune, or intervene in agent behaviors. The absence of persistent semantic memory further restricts agents from learning across sessions or leveraging past knowledge in meaningful ways, a shortcoming especially evident in single-session-only tools like ReAct and BabyAGI.

2.3 Comparative Analysis

This project addresses the shortcomings identified in existing systems by integrating the structured orchestration of LangGraph , the protocol-driven tool management of MCP , and the knowledge grounding provided by RAG (Vector Embeddings) . Table 2.1 provides a comparative analysis of approaches, highlighting the strategic integration used in this project.

Table 2.1: Comparison of Existing Approaches and Proposed Solution Features

Approach	Strengths	Limitations	Year	Core Method
AutoGPT / BabyAGI	Full autonomy, Zero-shot goal setting	Uncontrolled recursion, Goal drift, Opacity, Brittle prompting	2023	Task Chaining / Self-Prompting
LangChain Agents / ReAct	Tool use, Flexible library for connectors and components	Brittle prompt engineering, State persistence issues, Limited visualization	2022-2024	Tool Use, Chain of Thought
RAG [22, 7]	Reduced hallucination, Enhanced factuality (Knowledge Grounding)	Retrieval quality dependent on embedding and chunking, No agent orchestration	2020	Vector Search, Generator
LangGraph [35]	Structured, Cyclical workflows, State management, Inspectable execution	No inherent multi-agent communication protocol, Requires manual graph definition	2024	Graph-based State Machine
Model Context Protocol (MCP) [23, 17]	Structured context and tool discovery, Enhanced interoperability	Primarily a protocol, Not a complete execution framework (requires an agent host)	2025	Standardized Communication
Proposed Meta-Agent (LangGraph + MCP + RAG)	Transparent, Modular, Protocol-driven tool integration, Persistent Semantic Memory, Inspectable Execution	Increased initial setup complexity, Requires robust schema definition for tools	2025	Hybrid: Graph-driven Orchestration, Protocol-based Tooling, Semantic Memory

The proposed architecture overcomes the fragility, opaqueness, and rigidity of prior approaches. By utilizing LangGraph as the backbone, the system composes agent workflows as directed graphs, where each node represents a specialized sub-agent (e.g., Reasoner, Advisor, Coder, Refiner). This modular design allows agent behavior to be transparently constructed, debugged, and refined over time. It also supports deterministic and dynamic branching based

on intermediate results, ensuring that workflows remain both controllable and inspectable. Furthermore, the use of Supabase as a vector store introduces semantic memory into the system, allowing embeddings from prior conversations, documentation, or site data to be retrieved and used intelligently during reasoning and code generation. A key innovation is the integration of the Model Context Protocol (MCP) , which defines how agents can discover, evaluate, and invoke external tools based on formalized JSON schemas. This removes the hardcoded tool bindings seen in older systems and instead introduces a flexible, schema-driven approach where tools are treated as first-class citizens with documented inputs, outputs, and preconditions. This protocol-based communication ensures that agents operate within safe boundaries while enabling easier onboarding of new capabilities. The novelty of this work lies in its unification of visual agent orchestration, memory-based context grounding, protocol-driven tool invocation, and real-time human-guided refinement, all features absent or underdeveloped in earlier systems.

Chapter 3

SYSTEM SPECIFICATIONS

This chapter details the hardware and software requirements for implementing the project.

3.1 Software Requirements

- **Python 3.11** is chosen as the primary programming language for the system due to its simplicity, rich ecosystem, and compatibility with modern machine learning and agentic frameworks. Version 3.11 offers noticeable performance improvements and supports all required libraries. A virtual environment created using this version isolates dependencies and ensures reproducibility of the development environment.
- **Streamlit** is used to build the interactive user interface of the system. It enables rapid development of web-based dashboards with minimal code, allowing real-time interaction with agent responses. The interface runs on a local server and can be executed either natively or through Docker deployment.
- **FastAPI** acts as the core backend framework for hosting the Graph Service and managing interactions between the user interface and the agent workflows. It ensures high performance, supports asynchronous operations, and automatically generates OpenAPI-compliant documentation.
- **Docker** is used to containerize the complete system to ensure reliable deployment across environments. It encapsulates runtime dependencies, configurations, and services within lightweight containers. Docker Desktop is used during development, with support for both ARM and x86 architectures.
- **Supabase** functions as the vector database responsible for storing and retrieving document

embeddings. It is built on PostgreSQL and includes pgvector support, allowing efficient semantic similarity search. It provides secure access through API keys and authenticated endpoints.

- **Pydantic AI** enables type-safe definitions of agent models, tool schemas, and structured communication between components. It enforces data validation and preserves consistency across the entire workflow while supporting modular execution logic.
- **LangGraph** is used for multi-agent orchestration. It represents the workflow as a directed graph of interconnected reasoning, planning, and coding components, allowing conditional branching, error handling, iterative refinement, and transparent inspection of execution flow.
- **Playwright** is integrated for browser automation and retrieval of structured content from web sources during crawling workflows. A separate installation step is required to obtain browser binaries for local execution.
- **Model Context Protocol (MCP)** defines the structured mechanism for connecting agents with external tools, documents, and databases. It standardizes how context is shared and eliminates brittle hardcoded integrations by enabling reusable, schema-driven tool invocation.
- **OpenAI, Anthropic, and Ollama APIs** provide the language model backends that power reasoning, planning, and code generation. OpenAI and Anthropic rely on cloud-based API access, while Ollama supports running LLMs locally on supported hardware. Only one provider is required based on configuration.
- **pgvector** is the PostgreSQL extension that enables vector storage and similarity search for embeddings. It is automatically integrated through Supabase and forms the foundation of the system's semantic retrieval capability.
- **Git** is used to maintain the version control of the system and support team-based development. It allows tracking code updates, resolving conflicts, and maintaining collaboration history across repositories.
- **Rust Toolchain** is required to compile specific Python dependency packages such as tokenizers. It includes Cargo as the package manager and must be installed through the

rustup installer to support successful build operations on macOS and Linux.

- **Node.js (optional)** may be used for additional frontend tooling or JavaScript-based UI extensions. It is not mandatory for the core workflow but is helpful for interface customization when needed.
- **Operating System** compatibility includes Linux, Windows, and macOS. Linux is generally preferred due to its stability, scalability, and suitability for distributed computing workloads. It is widely adopted in machine learning and backend deployment environments.

3.2 Hardware Requirements

The system requires suitable hardware to ensure smooth execution of the user interface, backend services, vector database operations, and AI model processing. The hardware requirements are listed below:

- A computer used for normal development should have at least a **4-core CPU, 16 GB RAM, and 50 GB free storage**. This configuration is sufficient to run the user interface, execute backend services, and test small-scale agent workflows.
- For better performance and smoother multitasking, especially when processing large documents or interacting with AI tools, a **recommended configuration** includes an **8-core CPU, 32 GB RAM, and a 250 GB SSD/NVMe drive**. If local language models are used, a **dedicated GPU such as NVIDIA RTX 3070 or higher** significantly improves speed.
- For heavy workloads or production environments, such as supporting multiple users or running large AI models, a **high-end configuration** is needed. This may include a **server-grade multi-core CPU, 128 GB or more RAM, 1 TB or more high-speed NVMe storage, and professional GPUs such as NVIDIA A100**.
- **Apple Silicon systems** (M1, M2, M3) are fully compatible and work well for development and medium-scale workloads. They perform best with smaller optimized models, while large-scale production deployments are better suited to GPU servers.

- In all configurations, the hardware must support **Docker** and provide **sufficient memory and CPU allocation** to ensure stable execution of services, especially when running the vector database and agent workflows simultaneously.

Chapter 4

SYSTEM DESIGN

This chapter presents the architectural and design aspects of the system.

4.1 System Architecture

Figure 4.1 illustrates the end-to-end flow of data and control across the system, from user interaction to response generation, tool integration, and knowledge ingestion.

The following subsections describe the complete end-to-end flow of the system, beginning from user interaction and extending through agent orchestration, retrieval, tool invocation, reasoning, and offline data ingestion.

4.1.1 User Interaction Layer

- **User → Streamlit UI:** The user interacts with the system through a Streamlit-based front-end interface. This interface enables users to input queries, configure settings, and visualize agent outputs in real-time.

4.1.2 Initial Query Processing

- **Streamlit UI → Query Handler:** The input query from the user is passed to the query handler module, which performs preprocessing, session management, input validation, and task classification.
- **Query Handler → Meta-Agent:** Once validated, the processed query is forwarded to the Meta-Agent, which determines subsequent actions such as invoking tools, triggering RAG, or initiating code generation.

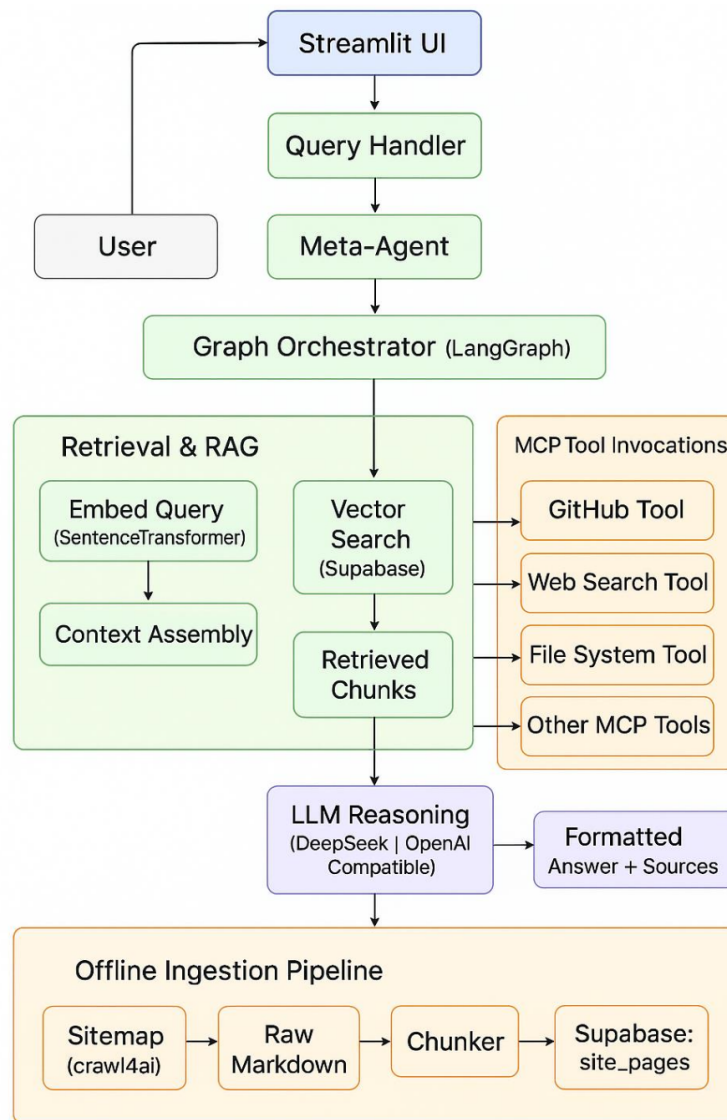


Figure 4.1: System Architecture Diagram

4.1.3 Agent Orchestration Layer

- **Meta-Agent → Graph Orchestrator (LangGraph):** The LangGraph-based Graph Orchestrator takes over execution by following a structured graph where each node represents a specialized agent or function. It coordinates reasoning, retrieval, tool calls, state transitions, and refinement loops.

4.1.4 Retrieval-Augmented Generation (RAG)

- **Embed Query (SentenceTransformer):** The user query is embedded into a numerical vector representation using a SentenceTransformer model.

- **Vector Search (Supabase):** The vector embedding is used to perform similarity search on the Supabase-hosted vector database, retrieving the most relevant document chunks.
- **Context Assembly:** Retrieved document chunks are assembled to form contextual memory, which is injected into the language model to improve grounded and factual generation.

4.1.5 Tool Invocations via MCP (Model Context Protocol)

- **MCP Tool Invocation Modules:** Based on decisions from the orchestrator, external tools may be invoked via MCP. Examples include:
 - GitHub Tool : for searching repositories, files, and code snippets.
 - Web Search Tool : for live internet search.
 - File System Tool : for handling local files or directories.
 - Additional MCP Tools : any registered tool with a valid manifest can be called, making the system extensible.

Each tool invocation is securely executed and returns structured data that contributes to the final response.

4.1.6 LLM Reasoning

- **LLM Reasoning (DeepSeek / OpenAI):** Using retrieved context and MCP tool outputs, the reasoning engine (e.g., DeepSeek, OpenAI) generates the final response. The result may include code, explanations, decisions, or task-specific instructions.
- **Formatted Output:** The generated response is formatted with citations or references and returned to the Streamlit UI for user visualization.

4.1.7 Offline Ingestion Pipeline

- This pipeline runs independently of live queries and updates the knowledge base for future retrieval.
- **Sitemap (crawl4ai):** Crawls websites based on sitemap data to collect relevant pages.
- **Raw Markdown:** Pages are scraped and converted into markdown for consistency.

- **Chunker:** Markdown files are chunked into smaller semantically meaningful segments.
- **Supabase (site_pages):** The chunks are embedded and stored in the Supabase vector database for future RAG retrieval.

4.2 Low-Level Design

The Low-Level Design (LLD) for the Graph-Driven Meta-Agents platform details the technical implementation of its core modules, agent orchestration logic, and integration mechanisms. This section provides a granular view of the system's architecture, focusing on the internal workflows, control structures, and extensibility features that underpin autonomous agent generation using Model Context Protocol (MCP) and LangGraph.

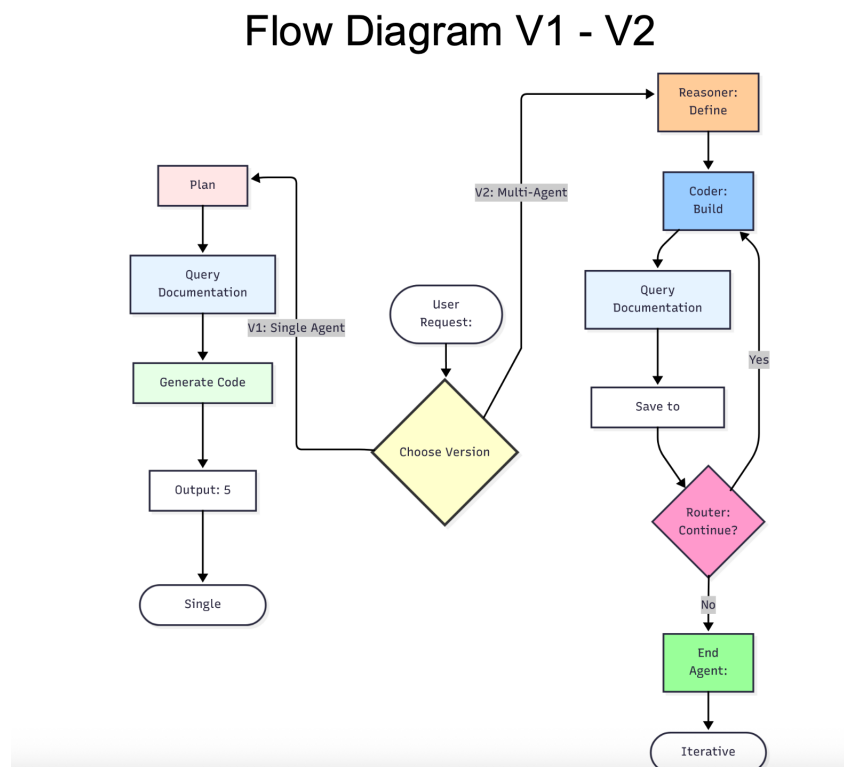


Figure 4.2: Low Level System Design

The following subsections describe the major components involved in the Low Level Design illustrated in Fig.4.2. Each component is designed to support modular, extensible, and fault-tolerant workflow execution for autonomous agent systems.

4.2.1 User Interface Layer

- The user interface is implemented using Streamlit, providing an interactive dashboard for agent configuration, workflow visualization, and result inspection.
- The UI communicates with backend services via RESTful API endpoints and direct Python function calls to ensure low-latency updates and real-time interaction.
- Input validation, error reporting, and session management are handled within the UI using Streamlit's stateful widgets and callback mechanisms.

4.2.2 Backend Services

- Backend logic is modularized into dedicated Python scripts and service classes.
- Each workflow type (single-agent, multi-agent, meta-agent) is encapsulated in its own module with strict separation of concerns.
- The backend exposes APIs for agent instantiation, documentation querying, code generation, and result aggregation.
- Asynchronous execution is supported using `asyncio` and `concurrent.futures`, enabling parallel agent execution and responsive UI feedback.

4.2.3 Graph-Based Agent Workflows

- Agent orchestration is represented as directed graphs in which nodes correspond to agent roles (e.g., Reasoner, Coder, Router).
- Edges encode control-flow rules, allowing dynamic runtime branching based on intermediate results or user instructions.
- LangGraph configuration files define graph topology and transitions.
- Each node is implemented as a Python class with standardized interfaces for input/output data, error handling, and state persistence.

4.2.4 External Tool Integration

- The system integrates with external documentation sources, repositories, and model APIs using adapter modules.
- Adapters abstract protocol details and provide unified interfaces for querying, parsing, and caching external data.
- The design is extensible—new tools or APIs can be added with minimal modifications to core logic, following the same adapter patterns.

4.2.5 Memory Storage and State Management

- Agent memory and intermediate outputs are stored using in-memory structures (dictionaries, lists) and optional persistent formats (SQLite, JSON files).
- The memory subsystem supports serialization, checkpointing, and retrieval to enable iterative workflows and recovery after failures.
- All data formats follow JSON-compatible schemas for interoperability across components.

4.2.6 Runtime Environment

- The system executes inside isolated Python virtual environments to ensure reproducibility and avoid dependency conflict.
- Dependencies are managed through `requirements.txt`, and configuration values are injected via `.env` files.
- The runtime supports both local and cloud deployment, with resource-management features for scaling agent runs and monitoring performance.

4.2.7 Error Handling and Concurrency

- Robust error handling includes try-except blocks, logging utilities, and informative user-facing error messages.
- Concurrency is managed using thread pools and asynchronous coroutines with safeguards against race conditions and deadlocks.

- In multi-agent workflows, the Router component determines continuation or termination based on runtime conditions, ensuring graceful degradation and automatic recovery.

4.2.8 Extension Points

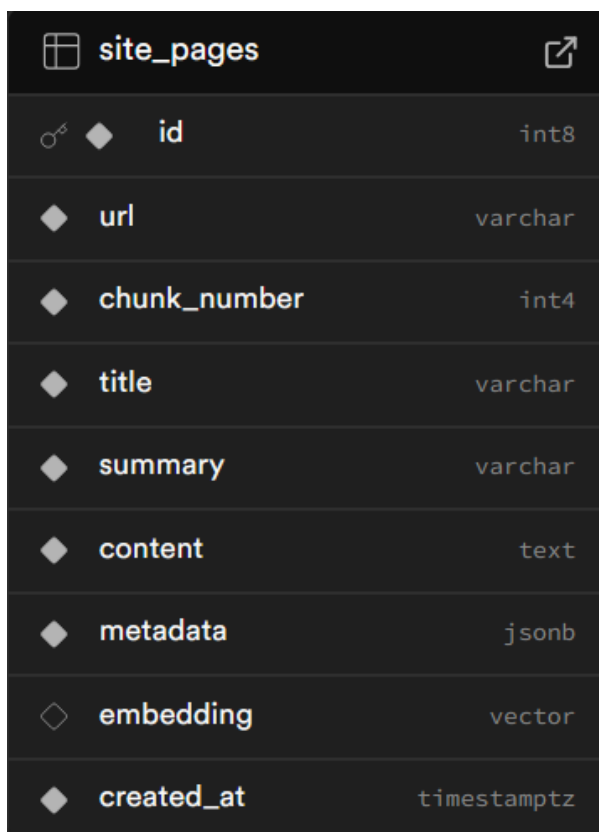
- The architecture is designed for long-term extensibility with clear interfaces for adding:
 - new agent types,
 - new workflow nodes,
 - new external integrations,
 - and new UI components.
- Modular organization and configuration-driven design enable rapid prototyping and customization for future contributors.

4.2.9 Database Schema Overview

The primary table in the database is `site_pages`, which stores chunks of documentation along with metadata and vector embeddings. The table schema is as follows:

As shown in Figure 4.3, the schema includes various fields for...

- **id**: A unique identifier for each chunk, implemented as a `bigserial` primary key.
- **url**: The URL of the documentation page, stored as a `varchar`.
- **chunk_number**: The chunk number within the page, stored as an `integer`.
- **title**: The title of the documentation page, stored as a `varchar`.
- **summary**: A brief summary of the chunk, stored as a `varchar`.
- **content**: The full content of the chunk, stored as `text`.
- **metadata**: Additional metadata for filtering, stored as `jsonb`.
- **embedding**: A vector column with 384 dimensions, used for similarity searches.
- **created_at**: A timestamp indicating when the chunk was created, stored as `timestampz`.

The image shows a database schema for a table named 'site_pages'. The table has 10 columns: 'id' (int8, primary key), 'url' (varchar), 'chunk_number' (int4), 'title' (varchar), 'summary' (varchar), 'content' (text), 'metadata' (jsonb), 'embedding' (vector), and 'created_at' (timestampz). Each column is represented by a diamond icon, and the 'id' column has a key icon indicating it is the primary key. The table name 'site_pages' is at the top left, and a share icon is at the top right.

site_pages	
id	int8
url	varchar
chunk_number	int4
title	varchar
summary	varchar
content	text
metadata	jsonb
embedding	vector
created_at	timestampz

Figure 4.3: Database Schema for site_pages table

4.2.10 Indexes and Constraints

To optimize performance and ensure data integrity, the following indexes and constraints are applied:

- A unique constraint on `url` and `chunk_number` to prevent duplicate chunks for the same URL.
- An `ivfflat` index on the `embedding` column for efficient vector similarity searches.
- A GIN index on the `metadata` column for faster filtering.

4.2.11 Search Functionality

The database includes a custom function, `match_site_pages`, to perform vector similarity searches. This function takes a query embedding, a match count, and an optional filter as inputs and returns the most similar documentation chunks based on cosine similarity.

4.2.12 Security Policies

Row-level security (RLS) is enabled on the `site_pages` table to ensure controlled access. The following policies are implemented:

- Public read access for selecting data.
- Public insert and update access for modifying data.

4.2.13 Embedding Update and Maintenance

The schema supports updates to the embedding column to accommodate changes in vector dimensions. For example, the embedding column was updated to 384 dimensions to align with the `sentence-transformers/all-MiniLM-L6-v2` model.

4.2.14 Entity Relationships and Data Flow

The `site_pages` table acts as a standalone entity, with no direct relationships to other tables. Data flows into the table through chunking and embedding processes, and retrieval is performed via vector similarity searches.

This schema design ensures scalability, efficient search performance, and flexibility for future enhancements.

Chapter 5

SYSTEM IMPLEMENTATION

This chapter presents the detailed description of all software modules used in the project. Each module contributes to the autonomous agent framework by handling user interaction, workflow orchestration, tool invocation, data retrieval, code generation, protocol execution, and runtime configuration.

5.1 Modules Used with Description

5.1.1 Streamlit UI Design

The user interface of the system is built using Streamlit and modular pages . It acts as an interactive workbench for configuring, launching, and monitoring agent workflows without code intervention.

Purpose and Functionality

The UI provides seven core tabs:

- Intro – Overview and setup status
- Chat – Real-time conversation with the agent
- Environment – View/edit environment variables (API keys, model selection, URLs)
- Database – Visual interaction with Supabase vector database
- Documentation – Crawling and ingestion of documents
- Agent Service – Start/Stop the Graph Service and create agents

- MCP – Manage Model Context Protocol integrations

Each tab is a separate Python module loaded dynamically from the main controller.

Backend Integration

The UI interacts with backend services via:

- POST requests to `/invoke` endpoint to start agent workflows
- File I/O for storing configuration
- Crawling documents through Playwright and indexing them in Supabase

Inputs and Outputs

User inputs include text fields, select boxes, file uploads, and button triggers. Configuration persistence is handled through:

Async Behavior and Error Handling

- Asynchronous streaming of messages for real-time chat
- UI-level validation for missing/invalid API keys or URLs
- Standardized error messages for Graph Service communication failures

Testing and Extensibility

- UI workflows tested using Playwright and pytest
- New pages added simply by extending re-registering

5.1.2 Graph Service Design (FastAPI)

The Graph Service is the backend API managing agent execution. It receives user input, triggers LangGraph workflows, and returns structured results.

Main Endpoints

- **GET /health** – Service operational check
- **POST /invoke** – Initiates or resumes agent conversation

Responses contain agent messages, generated artifacts, intermediate reasoning steps, and updated state.

Execution Behavior

- First message → start new workflow
- Follow-up message → resume from stored state
- Outputs aggregated and returned (non-streaming mode)

5.1.3 LangGraph Workflow Engine

The LangGraph engine models agent reasoning as a **StateGraph** with structured nodes and transitions. It supports branching, refinement loops, resumability, and fault tolerance.

Key Graph Nodes

- **define_scope_with_reasoner** – Parses intent and writes `scope.md`
- **advisor_with_examples** – Suggests relevant tools and documentation
- **coder_agent** – Generates executable code using RAG
- **refine_*** – Multiple refinement nodes for prompts, tools, code
- **route_user_message** – Determines next action based on intent
- **finish_conversation** – Summarizes and ends workflow

A `MemorySaver` module checkpoints state for resuming long workflows.

5.1.4 Pydantic AI Coder Component

Responsible for converting scoped instructions into executable code.

- **run()** for batch generation
- **run_stream()** for incremental streaming output
- Uses RAG to retrieve documentation from Supabase embeddings
- Outputs structured JSON containing generated code or diff blocks

5.1.5 Refiner Agents

Refiners iteratively improve intermediate outputs.

- **prompt_refiner_agent** – Refines task definition
- **tools_refiner_agent** – Filters and improves tool recommendations
- **agent_refiner_agent** – Adjusts reasoning or planning logic

5.1.6 Advisor Agent

Reads tool manifests in `agent-resources/` and produces a ranked list of tool suggestions using semantic relevance. Used downstream during code generation.

5.1.7 Agent Tools & Resources

Tools are declared using JSON manifests containing:

- `tool_name`, `entry_point`, `inputs`, `outputs`, `description`, `tags`

New tools can be added via manifest without code modification.

5.1.8 MCP Server & Integration

Supports secure, protocol-based execution of tools in external IDEs/runtimes using JSON messages.

5.1.9 Utilities & Database Integration

- `get_clients()` initializes Embedding and Supabase clients
- `site_pages.sql` defines vector storage schema for embeddings

5.1.10 Workbench Runtime Files

Files generated during runtime include:

- `env_vars.json` – Configuration
- `scope.md` – Task definition
- `logs.txt` – Execution logs

5.1.11 Docker Deployment

`run_docker.py` automates container lifecycle and networking. Ports exposed: 8501 (UI) and 8100 (Graph Service).

5.1.12 Configuration & Environment

Environment variables control provider access, Supabase connection, and model selection. Formats validated at startup.

5.1.13 APIs and Contracts

POST /invoke is the primary endpoint supporting:

- `is_first_message`
- `message`
- `state`
- `config`

5.1.14 Data Models

`AgentState` stores session memory; `ModelMessage` stores conversation history in JSON.

5.2 Concurrency, Async Behavior & Resource Management

To support real-time responsiveness and simultaneous agent sessions, the system incorporates asynchronous programming, concurrency control, and resource-aware execution across backend services.

- **Async Components:** The `graph_service` FastAPI routes are implemented using `async def`, enabling concurrent request handling. LangGraph relies on the `astream` function for asynchronous state transitions, while streaming from LLMs is managed using `async` iterators and callback-based writer mechanisms.
- **Streaming Implementation:** Responses are streamed using generator-style logic that yields partial text/code chunks incrementally. The UI begins displaying output before the model finishes generating the full response, improving responsiveness and user interaction quality.
- **Concurrency Limits:** The system runs on the `uvicorn` ASGI server with multiple worker processes to parallelize execution. The recommended worker count is proportional to CPU cores — typically 4–8 workers in development, with higher scaling in production deployments.
- **Backpressure and Slow LLM Handling:** When a language model or external API responds slowly, timeouts are enforced to prevent blocking. Stalled `async` operations are cancelled gracefully, and fallback logic reverts to the most recent stable state to ensure reliability and avoid infinite hangs or UI freezes.

Chapter 6

SYSTEM TESTING

This chapter describes the testing strategies and procedures used to validate the system.

6.1 Testing Strategy

This chapter describes the testing strategies and procedures used to validate the system, **Graph-Driven Meta-Agents: Integrating Model Context Protocol and LangGraph for Autonomous Agent Generation**.

6.2 Testing Strategy

The objective of the system testing plan was to validate the **correctness, performance, security, and overall functionality** of the entire architecture. The testing strategy spanned across various layers of the system, including the user interface (**Streamlit**), the **Graph Service** powered by **LangGraph**, the **Model Context Protocol (MCP)**, the backend components such as the vector database (**Supabase**), and the integration of language models and coders. Comprehensive testing was essential to ensure that the workflows behave as expected, resources are securely accessed, and the system remains stable under real-world usage conditions.

Testing was performed in three primary environments: **local development** using Python virtual environments, **containerized environments** using Docker, and **continuous integration** pipelines set up with GitHub Actions. These setups allowed for reliable replication of different real-world deployment scenarios. Test data and responses from external services such as LLMs were **mocked** to ensure deterministic and repeatable outcomes during test execution.

Types of Tests Conducted

The following types of tests were conducted to ensure system quality:

- **Unit Testing:** Focused on small and individual parts of the system, such as functions and logic components. For instance, functions handling environment variables, serializers, and small decision-making routines in the refiner modules were tested in isolation. Each unit test verified the internal logic, returned expected values, and handled invalid inputs gracefully.
- **Integration Testing:** Examined how well different components worked together. This included scenarios where various nodes within the LangGraph interacted with tools like the coder, advisor, and reasoner agents. Integration was also validated between the FastAPI-based Graph Service and the *agentic_flow* pipeline, confirming modules could exchange information and cooperate effectively.
- **End-to-End (E2E) Testing:** Conducted to simulate complete user journeys across the system, from initiating a conversation in the Streamlit interface to generating an agent and arriving at a final code output. These tests verified the full operation, combining UI behavior, backend logic, and database interactions.
- **API and Contract Testing:** Applied to the exposed RESTful endpoints, such as `GET /health` and `POST /invoke`. This ensured that the endpoints adhered to expected input and output formats, responded with the correct status codes, and gracefully handled errors such as timeouts or missing data.
- **Mocked LLM Testing:** Used mocked language model clients to return deterministic, controlled outputs in place of real APIs. This enabled the verification of system behavior under known conditions without relying on costly or unpredictable external API responses.
- **Database Testing:** Verified that document embeddings and metadata were properly inserted, queried, and retrieved in **Supabase**. Schema definitions (e.g., the *site_pages* table) were validated using SQL scripts.
- **Security and Privacy Testing:** Included validation of secrets handling, sandboxing of any code generated by agents, and testing input sanitization to prevent untrusted code execution.

- **Regression Testing:** Performed periodically using a test suite of known error scenarios to ensure that new code changes did not reintroduce previously resolved bugs.
- **User Acceptance and Usability Testing:** Focused on validating whether the system was intuitive, responsive, and aligned with user expectations by executing sample real user behaviors on the Streamlit interface.

Test Environment Configuration

The test environment was configured meticulously to ensure consistency:

- A **Python 3.11 virtual environment** was created with all dependencies installed from *requirements.txt*.
- **Docker** was used to containerize the Graph Service and MCP for consistent test deployment.
- A dedicated **Supabase test database** or an emulated local Postgres database was configured for isolated data testing.
- **Playwright** was installed to automate UI interaction tests.
- **Mock services for LLMs** were configured to simulate real-time outputs without external calls.

Fixtures and Test Data

Test data included pre-written user requests and documentation chunks. Embedding vectors were seeded with fixed small values to allow consistent retrieval checks. **Pytest fixtures** ensured that test databases were populated before execution and cleaned up afterward.

6.3 Test Cases

A comprehensive suite of test cases was executed across the various testing layers (Unit, Integration, E2E). Table 6.1 illustrates a sample of the key test cases used to validate core system functionality.

Table 6.1: Sample Test Cases

Test ID	Description	Expected Result	Status
TC01	Verify data loading and chunking into <i>site_pages</i> table.	Document embeddings and metadata loaded successfully.	Pass
TC02	Test LangGraph flow with Reasoner → Coder → Refiner agents (Mocked LLM).	Correct code prediction/output generated and refined as per prompt logic.	Pass
TC03	Validate API <i>POST/invoke</i> output format (JSON schema adherence).	Output in specified JSON format with required status code (200).	Pass
TC04	Test semantic retrieval from Supabase with a known query.	Relevant documentation chunk successfully recalled via vector query.	Pass
TC05	E2E: Simulate user creating and exporting a simple agent from Streamlit UI.	Full user experience is smooth, final code is exportable.	Pass
TC06	Security: Test input sanitization against harmful prompt injection.	System does not execute untrusted code; generates a strong warning/error.	Pass

6.4 Performance Testing

Performance tests were conducted using tools like **locust** and **k6** to simulate multiple concurrent users invoking the Graph Service. These tests measured critical metrics to ensure the system's readiness for real-world deployment.

6.4.1 Performance Metrics

- **Latency (Response Time):** The system maintained a response time **under 2 seconds** in mocked environments. Bottlenecks were identified by gradually increasing the number of simulated users.
- **Throughput:** Measured the number of concurrent requests the system could handle stably. The system scaled up to **dozens of concurrent users** with stable performance.
- **Memory Usage and CPU Consumption:** Monitored during load tests to identify re-

source leaks or over-utilization.

- **Streaming & Stability Testing:** Streaming responses from LLM providers were tested for ordering, completeness, and error handling. Simulated failures mid-stream were introduced to validate error logging and service recovery, ensuring reliable output for the user.

6.4.2 CI and Automation

A **GitHub Actions workflow** was configured to run all test suites on each push to the repository. Matrix builds tested multiple Python versions, and test reports were published as artifacts. Linters and type checkers were included for code quality assurance, ensuring that legacy functionality remained stable across iterative improvements.

6.4.3 Security Measures

Security validation confirmed that API keys were **stored securely** and sensitive data never appeared in logs. Generated code snippets were **never auto-executed** in the sandbox, and strong warnings were shown to the user during export, upholding security and privacy standards.

Through comprehensive testing across all layers and scenarios, the system achieved high reliability, modular correctness, secure processing, and smooth user experience. This testing infrastructure remains critical for future development and expansion of the autonomous agent generation framework.

Chapter 7

RESULTS AND ANALYSIS

This chapter presents the results obtained from the evaluation of the Retrieval-Augmented Generation (RAG) system and provides a comprehensive analysis of its performance.

7.1 Experimental Setup

7.1.1 System Implementation

The RAG system was implemented using a Streamlit-based user interface that provides an intuitive platform for querying the knowledge base and receiving AI-generated responses. Figure 7.1 shows the main interface of the application, which features a clean, modern design with a navigation sidebar for easy access to different sections including Introduction, Chat, Environment configuration, Database setup, and Documentation management.

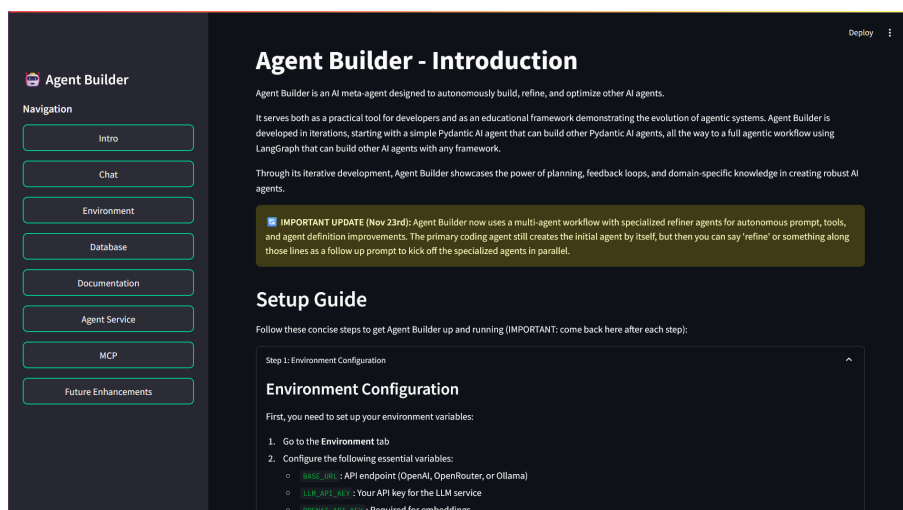


Figure 7.1: Streamlit UI - Main interface showing Agent Builder introduction and setup guide

The interface provides comprehensive setup instructions, guiding users through the config-

uration process. As illustrated in Figure 7.2, the system requires configuration of essential environment variables including API endpoints, authentication keys, and model specifications. The setup process is organized into sequential steps covering environment configuration, database setup, documentation crawling, agent service setup, and MCP configuration.

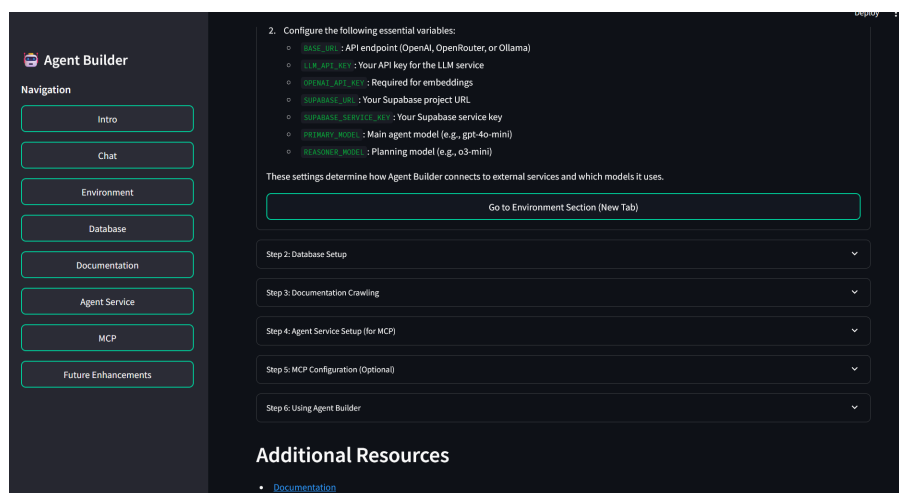


Figure 7.2: Streamlit UI - Environment configuration section showing required variables and setup steps

The system architecture integrates multiple components including:

- **Base URL and API Keys:** Configuration for OpenAI, OpenRouter, or Ollama endpoints with corresponding API keys for LLM services and embeddings
- **Database Integration:** Supabase project URL and service key for vector storage
- **Model Selection:** Configurable primary agent model (e.g., gpt-4o-mini) and planning model (e.g., o3-mini) for different reasoning tasks

7.1.2 Dataset and Knowledge Base

The evaluation was conducted using a knowledge base populated exclusively with Pydantic AI official documentation. The documentation was processed and stored in a vector database following the methodology described in Chapter 3. The knowledge base included:

- Introduction and overview sections
- Model definition and usage guidelines
- Validation features and examples

- Serialization and deserialization methods
- Error handling strategies
- Advanced configuration options

The documentation was crawled and processed through the system's documentation management interface, which automated the extraction, chunking, and vectorization of content from the official Pydantic AI documentation website.

7.1.3 Test Queries

A total of 5 representative queries were designed to cover different aspects of the documentation, ranging from basic introductory questions to advanced technical topics. These queries were selected to represent typical user inquiries in a developer support scenario and were tested through the chat interface shown in the system implementation.

7.1.4 Evaluation Metrics

The system performance was evaluated based on the following criteria:

- **Coverage:** Whether the retrieved chunks fully or partially covered the expected ground truth content (categorized as "Full" or "Partial")
- **Relevance:** The semantic relevance of retrieved chunks to the query
- **Accuracy:** The correctness of information in retrieved chunks
- **Completeness:** Whether all necessary information was retrieved to answer the query comprehensively

7.2 Results

7.2.1 Accuracy and Relevance

To evaluate the accuracy and relevance of our RAG system, we conducted systematic testing using the prepared queries through the Streamlit interface. Table 7.1 summarizes the query performance against expected ground truth.

Query	Expected Context (Ground Truth)	Retrieved Chunks	Coverage	Comments
What is Pydantic AI?	Introductory section describing Pydantic AI, its purpose, and main features.	Chunk 2: “Pydantic AI is a library for ...” Chunk 3: “It enables ...”	Full	All relevant introductory information retrieved.
How do you define a model in Pydantic AI?	Example code and explanation from the “Model Definition” section.	Chunk 47: “To define a model, use ...” Chunk 48: “class User(BaseModel): ...”	Full	Correct code sample and explanation included.
What validation features does Pydantic AI provide?	List and description of validation features from the “Validation” section.	Chunk 89: “Pydantic AI supports ...” Chunk 91: “You can use validators ...”	Partial	Missed advanced validation options; basic features retrieved.
How to serialize a model?	Steps and code for serialization from the “Serialization” section.	Chunk 134: “Models can be serialized using ...”	Full	Serialization process clearly explained.
How to handle errors in Pydantic AI?	Error handling strategies and code samples from the “Error Handling” section.	Chunk 167: “Error handling in Pydantic AI ...”	Partial	Only general error handling retrieved; specific examples missing.

Table 7.1: Query performance against expected ground truth

7.2.2 Example Query and Output

To illustrate the system’s performance in detail, we present a complete example of one test query, including the expected context, actual retrieved chunks, and analysis.

Query: “How do you define a model in Pydantic AI?”

Expected Context:

“To define a model, use the BaseModel class. For example:

```
class User(BaseModel):
    id: int
    name: str
```

Actual Retrieved Chunks:

“To define a model, use the BaseModel class.”

“class User(BaseModel): id: int; name: str”

Analysis:

The system successfully retrieved the relevant code sample and explanation, demonstrating high accuracy for this query. The retrieved chunks contained both the conceptual explanation and the concrete code example, allowing the LLM to generate a comprehensive response. This example illustrates the system’s effectiveness in handling queries that require both textual explanation and code samples.

7.3 Analysis

7.3.1 Performance Summary

The evaluation revealed that the RAG system achieved full coverage for 60% of the test queries (3 out of 5), with the remaining 40% showing partial coverage. This indicates strong overall performance, particularly for well-documented topics with clear examples.

7.3.2 Key Findings

The evaluation of our RAG system revealed several key insights:

- **High Accuracy for Basic Queries:** The RAG system consistently retrieved relevant documentation chunks for introductory and fundamental usage queries, with full coverage achieved in 60% of test cases. This demonstrates the system’s reliability for common developer inquiries.
- **Strong Performance on Code Examples:** Queries requesting code samples (e.g., model definition, serialization) received accurate and complete responses, indicating effective chunking and indexing of code-related content. The system successfully preserved code structure and context during the chunking process.
- **Partial Coverage for Advanced Topics:** Some advanced topics, such as custom validators and detailed error handling strategies, were only partially covered. This suggests opportunities for improvement in chunk granularity or the need for more sophisticated retrieval strategies for complex queries.

- **Effective Semantic Understanding:** The embedding model demonstrated good semantic understanding, successfully matching queries to relevant documentation sections even when exact keyword matches were not present. This highlights the advantage of using semantic embeddings over traditional keyword-based search.
- **Documentation-Specific Performance:** The results validate the system's ability to support developer queries using official documentation as the knowledge base, making it suitable for technical support and developer assistance applications.
- **User Interface Effectiveness:** The Streamlit-based interface (Figures 7.1 and 7.2) provided an intuitive platform for system configuration and query interaction, facilitating easy testing and evaluation of the RAG system's performance.

7.3.3 Limitations and Areas for Improvement

While the system demonstrated strong performance overall, several limitations were identified:

- **Incomplete Coverage of Advanced Topics:** Complex queries requiring multiple related concepts showed partial coverage, suggesting the need for improved chunk linkage or context preservation.
- **Chunk Granularity:** Some topics may benefit from different chunking strategies to ensure complete information retrieval without sacrificing relevance.
- **Limited Test Coverage:** The evaluation was based on a small set of representative queries. A larger and more diverse and real world test set would provide more comprehensive insights into system performance.

7.3.4 Implications for Deployment

These findings indicate that while the RAG system performs well for common use cases, there is room for enhancement in handling advanced or edge-case scenarios. Future improvements could include:

- Refining the chunking strategy for complex topics to maintain context while ensuring complete coverage

- Implementing query expansion techniques to improve retrieval for ambiguous or under-specified queries
- Incorporating a re-ranking mechanism to improve retrieval precision for partial matches
- Adding a feedback loop to continuously improve retrieval based on user interactions
- Enhancing the user interface with additional features such as query history, relevance feedback, and performance metrics visualization

The current system demonstrates sufficient accuracy and reliability for deployment in developer support scenarios, with ongoing monitoring and refinement recommended to address the identified limitations.

Chapter 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The work titled Graph-Driven Meta-Agents: Integrating Model Context Protocol and LangGraph for Autonomous Agent Generation presents a transformative approach to building intelligent, modular agents capable of performing complex workflows with autonomy and contextual awareness. Unlike traditional monolithic agents that are limited to fixed tools or linear tasks, this system introduces a scalable meta-agent architecture that uses LangGraph to model agent reasoning as graphs and MCP to securely interact with external tools, documents, and databases.

To ensure steady development, testing, and scalability, the project was deliberately structured into multiple versions (V1 through V6). This version-based progression allowed for a clear roadmap where each stage could introduce, test, and stabilize key features incrementally. It enabled the team to isolate technical challenges, validate components at each layer, and build on proven functionality, ensuring that complexity grew in a manageable and maintainable way. Each version served as a controlled milestone, with specific goals and improvements leading to the mature and robust meta-agent system developed.

Throughout the development process, the system has matured through six structured versions, each adding new capabilities, robustness, and real-world applicability:

- **V1** introduced the basic integration between a single agent and LangGraph. It demonstrated simple reasoning tasks and showcased how LangGraph could model the flow between reasoning, acting, and observing.
- **V2** expanded the system to support multiple roles through agent role chaining. Agents were able to delegate tasks and collaborate across structured roles, enabling early multi-agent behavior.

- **V3** integrated external tool access using the **Model Context Protocol (MCP)**. This allowed agents to securely interact with tools like Supabase and document retrieval APIs by following a standardized communication pattern for accessing contextual information.
- **V4** transformed the framework from single-purpose agents to a complete **meta-agent architecture**. The meta-agent could dynamically plan tasks, spawn specialized sub-agents with different toolkits, and evaluate the outcomes of delegated tasks, greatly improving scalability and flexibility.
- **V5** introduced a **Streamlit-based user interface**, providing a workbench where users could visualize the planning graph, inspect agent reasoning, and configure model and tool parameters through an accessible dashboard. This version significantly enhanced usability and transparency for non-technical users.
- **V6**, the most advanced version, integrates planner and executor loops with support for **parallel sub-agent execution, retry and error handling**, and a built-in **memory mechanism**. These improvements enable stable long-term planning, self-correction, and make the system suitable for real-world deployment in coding and knowledge-work applications.

Each version incrementally added to the system's modularity, intelligence, and autonomy. Together, they reflect a strong foundation not just for a coding assistant, but for a generalized, extensible meta-agent system that can work across different domains and toolchains.

In terms of societal relevance, this work aligns with UN SDG Goal 9 (Industry, Innovation, and Infrastructure) by laying the groundwork for intelligent software systems that can optimize workflows and reduce manual cognitive effort. It also supports Goal 4 (Quality Education) by enabling more accessible and interactive learning through agent-guided explanations, code generation, and document retrieval.

In conclusion, this project demonstrates a highly promising evolution toward next-generation AI systems. Through its structured graph-based design, modular tooling, and continuous version upgrades, it not only solves real-world technical problems but also creates opportunities for future innovation in agent collaboration, AI safety, and developer productivity. The groundwork established in V1–V6 prepares the system for further research, fine-tuning, and eventual integration into enterprise, academic, and open-source ecosystems.

8.2 Future Enhancements

The development of Graph-Driven Meta-Agents: Integrating Model Context Protocol and LangGraph for Autonomous Agent Generation has laid a strong foundation for modular, context-aware, and scalable AI systems. While the current system successfully demonstrates multi-agent collaboration, secure tool access, and flexible planning capabilities, several opportunities remain advancing its intelligence, usability, and real-world impact.

One key future direction is the integration of agent memory and long-term context tracking. Currently, agents operate in isolated task sessions, but introducing persistent memory would allow them to recall previous interactions, adapt to user preferences, and build knowledge over time. This enhancement would greatly benefit long-running workflows, project-based automation, and knowledge-centric domains such as research and education.

The system can also evolve to support fine-tuned agent specialization. Instead of relying solely on general-purpose agents, future versions could include agents trained or optimized for specific domains such as finance, law, medicine, or scientific research. These agents could be tailored with domain-specific tools, reasoning strategies, and language models, improving accuracy, relevance, and reliability in specialized use cases.

Another important future enhancement is scaling agent deployment to cloud-native environments. Currently, the system supports local and containerized operation, but future work could involve deploying agents across distributed platforms using Kubernetes or serverless infrastructure. This would allow dynamic resource allocation, high availability, and concurrent handling of multiple users or workloads, supporting real-world enterprise and SaaS deployment scenarios.

Toolchain expansion and inter-agent protocol standardization also present valuable growth paths. While MCP already provides a secure and extensible communication framework, future versions could incorporate broader APIs, such as real-time data streams, external automation services (e.g., Zapier, GitHub Actions), and human-in-the-loop interfaces. Standardizing these interfaces would allow interoperability with other autonomous agent systems and open collaborative workflows across different platforms.

Additionally, intelligent self-debugging and learning mechanisms can be explored. By allowing agents to assess their own outputs, detect inconsistencies, and refine their planning graphs, the system could become increasingly autonomous and reliable. This aligns with the

goal of creating self-improving systems that minimize human oversight.

From a user interaction perspective, the Streamlit-based UI could be extended to include natural language configuration, visual graph editing, and role-based dashboards, making it easier for non-technical users to interact with, configure, and monitor agents.

Finally, the system has strong potential to support broader societal goals. It can be used to develop assistive agents for education, such as tutors that explain complex subjects or help students with project planning. It can also enhance productivity tools for developers, researchers, and analysts, reducing cognitive load, and improving output quality.

In conclusion, the system's architecture is designed to be extensible and future-proof. Its modular design, secure context handling, and graph-based logic enable a wide range of innovations in multi-agent collaboration, real-time automation, and domain-aware intelligence. These advancements will continue to push the boundaries of what autonomous AI systems can achieve, both in research and in practical deployments.

8.3 Applications

The proposed graph-driven meta-agent framework opens the door to a wide range of real-world applications across software engineering, enterprise automation, research, and education. By combining structured orchestration (LangGraph), documentation-aware retrieval (Supabase + pgvector), tool interoperability (MCP), and self-refinement loops, the system moves beyond traditional chat-based assistants and enables autonomous AI workflows. The versatility and modularity of the architecture allow it to adapt to multiple domains with minimal reconfiguration.

- **Automated Software Development** : The meta-agent can autonomously generate, debug, refactor, and refine software code, reducing manual engineering effort and accelerating development life cycles.
- **Enterprise Knowledge Assistants** : Organizations can deploy internal assistants capable of retrieving information from documents, wikis, service portals, and databases using RAG + MCP-enabled tool usage.
- **Technical Documentation Generation** : The system can convert APIs, repositories, and architectural artifacts into well-structured documentation, helping teams maintain up-to-date technical manuals with minimal effort.

- **Autonomous Data and Web Research** : Through integrated web-scraping and summarization tools, agents can crawl websites, extract structured data, cross-verify sources, and produce concise research outputs.
- **Automated API and Workflow Orchestration** : The meta-agent can coordinate external services and tools via standardized schemas, creating reproducible and explainable automated workflows across platforms.
- **Education and Personalized Learning** : AI-driven tutoring modules can generate personalized explanations, learning paths, and hands-on examples tailored to a learner's profile and preferences.
- **Business Process Automation** : Routine processes such as KPI reporting, analytics retrieval, CRM updates, compliance monitoring, and document summarization can be handled automatically by domain-aware agents.
- **Customer Support Automation** : Agents can answer customer queries by searching documentation and product databases dynamically, enabling contextual and real-time support without manually created scripts.
- **Research and Scientific Workflows** : The system can assist in literature review, dataset preparation, annotation, and reproducible experiment execution using persistent memory and workflow branching.
- **Cybersecurity and System Monitoring** : Agents can monitor log data, detect anomalies, crawl vulnerability sources, and trigger predefined security workflows using MCP-validated tools.

Bibliography

- [1] L. Wang et al., “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [2] T. Guo et al., “Large language model based multi-agents: A survey of progress and challenges,” *arXiv preprint arXiv:2402.01680*, 2024.
- [3] A. Amato et al., “Multi-agent reinforcement learning for distributed workflow orchestration at the tactical edge,” in *Proc. MILCOM 2024 - IEEE Military Communications Conference*, 2024.
- [4] C. Yu et al., “A survey on agent workflow – status and future,” in *Proc. 2025 8th International Conference on Artificial Intelligence and Big Data (ICAIBD)*, 2025.
- [5] M. B. Blake, “Coordinating multiple agents for workflow-oriented process orchestration,” *Information Systems and e-Business Management*, vol. 1, pp. 387–404, 2003.
- [6] S. Kukreja et al., “Performance Evaluation of Vector Embeddings with Retrieval-Augmented Generation,” in *2024 9th Int. Conf. on Computer and Communication Systems (ICCCS)*, pp. 333–340, 2024.
- [7] B. Tural et al., “Retrieval-Augmented Generation (RAG) and LLM Integration,” in *2024 8th International Symposium on Innovative Approaches in Smart Technologies (ISAS)*, 2024.
- [8] D. Inani et al., “Smart University Assistant: Leveraging Graph Databases and Vector Embeddings for Efficient Information Retrieval,” in *2025 IETC*, 2025.
- [9] A. Sarnikar, “Using LLMs for Querying and Understanding Long Legislative Texts,” in *2025 IEEE SCEECs*, 2025.

- [10] A. S. et al., “Multi-agent hierarchical workflow for autonomous code generation with Large Language Models,” in *2025 IEEE SCEECs*, 2025.
- [11] R. Ramírez-Rueda et al., “Transforming Software Development: A Study on the Integration of Multi-Agent Systems and Large Language Models,” in *CONISOFT*, 2024.
- [12] M. V. Alekseyuk et al., “Development and Optimization of a Semantic Search System for Scanned Legal Documents,” in *2025 XXVIII SCM*, 2025.
- [13] I. Mahmood et al., “AzureIQ-RAG: Hybrid RAG System for Automated Information Retrieval and Semantic Query Answering,” in *2025 ICIT*, 2025.
- [14] A. Kotiyal et al., “Chat with PDF using LangChain Model: A Critical Review,” in *ICAIT-2024*, 2024.
- [15] S. Joshi, “Review of Autonomous Systems and Collaborative AI Agent Frameworks,” *Int. J. of Science and Research Archive*, 2025.
- [16] Z. Deng et al., “Exploring DeepSeek: A Survey on Advances, Applications, Challenges and Future Directions,” *IEEE/CAA Journal of Automatica Sinica*, vol. 12, no. 5, pp. 872–893, 2025.
- [17] J. Ahn et al., “An Agentic Model Context Protocol Framework for Medical Concept Standardization,” *UTHealth Mayo Clinic*, 2025.
- [18] Y. Bei et al., “Graphs Meet AI Agents: Taxonomy, Progress, and Future Opportunities,” *arXiv:2506.18019*, 2025.
- [19] O. Khattab et al., “DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines,” *Stanford, CMU, Microsoft*, 2025.
- [20] K. Opsahl-Ong et al., “Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs,” *arXiv:2406.11695*, 2024.
- [21] S. Bhupathi, “Building Scalable AI-Powered Applications with Cloud Databases,” *arXiv:2504.18793*, 2025.
- [22] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *NeurIPS*, 2020.

- [23] N. Krishnan, “Advancing Multi-Agent Systems Through Model Context Protocol,” *arXiv:2504.21030*, 2025.
- [24] R. Odobesku et al., “Agent-based Multimodal Information Extraction for Nanomaterials,” *npj Computational Materials*, vol. 11, p. 194, 2025.
- [25] F. Piccialli et al., “AgentAI: A Comprehensive Survey on Autonomous Agents in Distributed AI,” *Expert Systems with Applications*, vol. 291, p. 128404, 2025.
- [26] X. Zhang et al., “A Survey of Multi-AI Agent Collaboration,” in *DEAI 2025*, pp. 1–7, 2025.
- [27] P. P. Ray, “A Review on Agent-to-Agent Protocol,” *TechRxiv*, 2025.
- [28] R. Sharma et al., “Collaborative Agentic AI Needs Interoperability Across Ecosystems,” *arXiv:2505.21550*, 2025.
- [29] M. Abou Ali and F. Dornaika, “Agentic AI: A Comprehensive Survey,” *arXiv:2510.25445*, 2025.
- [30] R. Sapkota et al., “LangChain vs. LangGraph vs. LangSmith,” *TechRxiv*, 2025.
- [31] H. Derouiche et al., “Agentic AI Frameworks: Architectures, Protocols, and Design Challenges,” *IEEE*, 2025.
- [32] D. B. Acharya et al., “Agentic AI: Autonomous Intelligence for Complex Goals,” *IEEE Access*, vol. 13, pp. 18912–18940, 2025.
- [33] J. Fang et al., “A Comprehensive Survey of Self-Evolving AI Agents,” *arXiv:2508.07407*, 2025.
- [34] Y. Liu et al., “Graph-Augmented Large Language Model Agents,” *arXiv:2507.21407*, 2025.
- [35] J. Wang and Z. Duan, “Agent AI with LangGraph,” *arXiv:2412.03801*, 2024.
- [36] J. Tang et al., “AutoAgent: A Fully-Automated and Zero-Code Framework for LLM Agents,” *arXiv:2502.05957*, 2025.